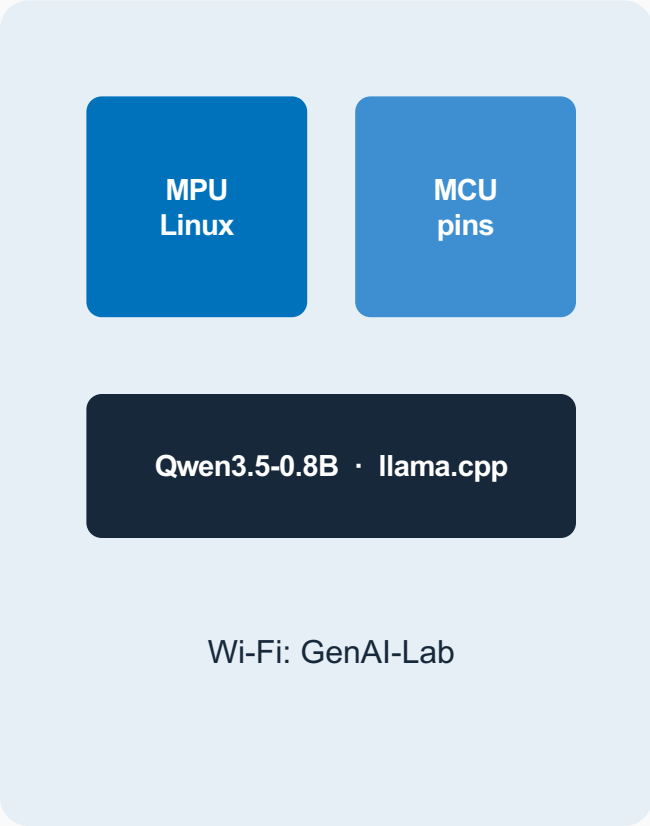
GenAI at the Edge · Student Lab

Chat with an AI that lives on an Arduino

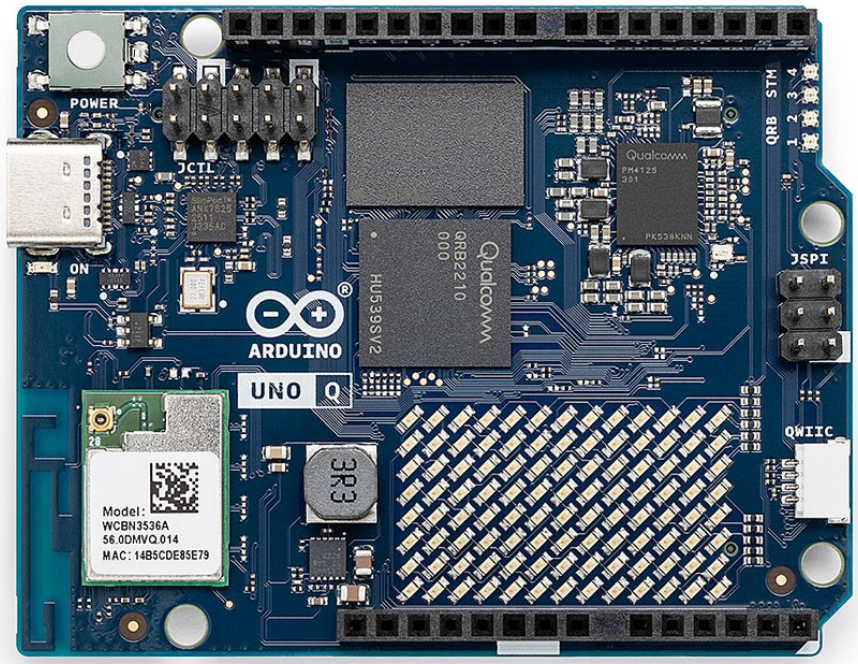
A small language model (Qwen3.5-0.8B) runs fully offline on the Arduino UNO Q at the front of the room. Your laptop talks to it over Wi-Fi.



- Laptop with Wi-Fi
- Any web browser
- Python (optional, part 2)

Based on “GenAI at the Edge” — Prof. Marcelo Rovai (UNIFEI)

Meet the Arduino UNO Q — a Linux computer and an Arduino on one board



Classic UNO size and headers, with a full Linux computer added on board.

Component	Specification
Linux processor	Qualcomm QRB2210, 4× A53 @ 2.0 GHz
Microcontroller	STM32U585, Cortex-M33 @ 160 MHz
RAM	2 GB or 4 GB LPDDR4X
Storage	16 GB or 32 GB eMMC, no SD card
Software	Debian Linux + Arduino on Zephyr
Wireless	Wi-Fi 5 (2.4/5 GHz), Bluetooth 5.1
USB-C	Power, host/device, 1080p video out
Power input	5 V / 3 A via USB-C, or 7–24 V VIN
I/O	UNO headers, I2C, SPI, UART, CAN, ADC
Size	68.6 × 53.3 mm

Nicla Vision has the sensors built in and draws a fraction of the UNO Q's power

	Arduino UNO Q	Nicla Vision
Processor	QRB2210, 4× A53 @ 2.0 GHz + STM32U585 MCU	STM32H747, M7 @ 480 MHz + M4 @ 240 MHz
Software	Debian Linux + Arduino on Zephyr	No OS: Arduino, MicroPython, OpenMV
Memory	2–4 GB RAM, 16–32 GB eMMC	1 MB RAM, 2 MB flash + 16 MB QSPI
Camera	None on board	**2 MP color camera (GC2145)**
Motion sensor	None on board	**6-axis IMU (accelerometer + gyroscope)**
Microphone	None on board	**PDM MEMS microphone**
Distance sensor	None on board	**Time-of-flight sensor, up to 4 m**
Wireless	Wi-Fi 5, Bluetooth 5.1	Wi-Fi 4, Bluetooth LE
Power, active	1.5–3.5 W under load	**≈0.4 W during image capture**
Power, idle	≈0.5 W, Linux always running	**≈1.4 mW in deep sleep**
Battery	No; needs a 5 V / 3 A supply	**3.7 V Li-Po connector with fuel gauge**
Size	68.6 × 53.3 mm	22.9 × 22.9 mm

Power: UNO Q measured at the wall with Wi-Fi on (Jeff Geerling, Oct 2025; sbc.compare). Nicla Vision from 105 mA during image capture and 374 µA in deep sleep on a 3.7 V Li-ion battery. Source: Arduino datasheets ABX00162 and ABX00051 (2025).

Step 1 — Join the board's Wi-Fi

1. Open your laptop's Wi-Fi menu and join the network below. Your phone works too.

2. Ignore the “no internet” warning — that is the point. Everything runs on the board.

3. Open a browser and go to the address on the right. You should see a chat page.

NETWORK

ClassroomAI

PASSWORD

louk-mele-wips-95

THEN OPEN

`http://10.42.0.1:8080`

If the page will not load: forget other Wi-Fi networks, confirm you are on the right WiFi, and retype the address exactly — it is http, not https.

Step 2 — Explore the chat interface

The page at 10.42.0.1 is llama.cpp's built-in GUI — a mini ChatGPT served by the Arduino.

1

Chat

Ask short, concrete questions. The model answers in real time — you will see tokens appear one by one (~5 per second).

2

Vision

Click the attach icon to upload a photo and ask about it. Keep images under 640×640 pixels; expect about a minute per image.

3

Take turns

One board, one request at a time. If it feels stuck, a classmate's request is probably running — watch the queue.

4

Good tasks

Classification, extraction, short explanations. Example: “Temp 30 °C, humidity 85%, standing water — mosquito risk? Answer yes/no + one reason.”

What is actually running? (spoiler: not the cloud)

Arduino UNO Q — a dual-brain board

MPU — the thinker
Qualcomm QRB2210
4× A53 cores, Debian Linux
runs llama.cpp + the model

MCU — the doer
STM32U585
real-time pins, LEDs, sensors
classic Arduino sketches

Two models to choose from, both text + images, 201 languages, offline:

- **Qwen3.5-0.8B (8-bit):** 0.8B parameters, ~1.1 GB of RAM, ~5 tok/s.
- **Qwen3.5-2B:** 2B parameters, 2.5× larger. Better answers, slower.

"Is this Ollama?" — close cousin.

Ollama is the popular tool for running local LLMs on laptops and Raspberry Pi — easy, but too heavy for this board.
llama.cpp is the lightweight engine underneath that world. It fits the UNO Q and serves the same OpenAI-style API — so everything you learn here transfers directly to Ollama and to cloud APIs.

3.5 W	~5 tok/s	100%
peak power — no fan	generation speed	offline & private

Same recipe scales up: your phone's AI assistant and data-center LLMs work exactly this way — just bigger.

Step 3 — Use it from Python

The board speaks the standard OpenAI API. Stay on GenAI-Lab Wi-Fi and run:

```
# pip install openai
from openai import OpenAI

client = OpenAI(
    base_url="http://10.42.0.1:8081/v1",
    api_key="none")    # any string works

resp = client.chat.completions.create(
    model="qwen",
    messages=[{"role": "user", "content":
        "Mosquito risk at 30C, 85% humidity?"}],
    max_tokens=150)

print(resp.choices[0].message.content)
```

Same code, three worlds

Swap base_url and this exact script talks to Ollama on your laptop or to cloud APIs.

Vision too

Send a base64 image as an image_url content part — the standard OpenAI vision format.

Share the board

Requests queue one by one. Keep max_tokens small in class.

Your turn — three challenges

1

Prompt engineer

In the web GUI, get the model to answer a yes/no classification with exactly one sentence of reasoning. How reliable is it over 5 tries?

2

Vision test

Upload a photo of something on your desk. Where does the model do well — and where does it hallucinate details?

3

Python benchmark

From Python, time 3 requests and estimate tokens/second. Compare with the ~5 tok/s we promised.

Remember: the whole thing runs on 3.5 watts with no internet. Small models, honest limits — keep a human in the loop.